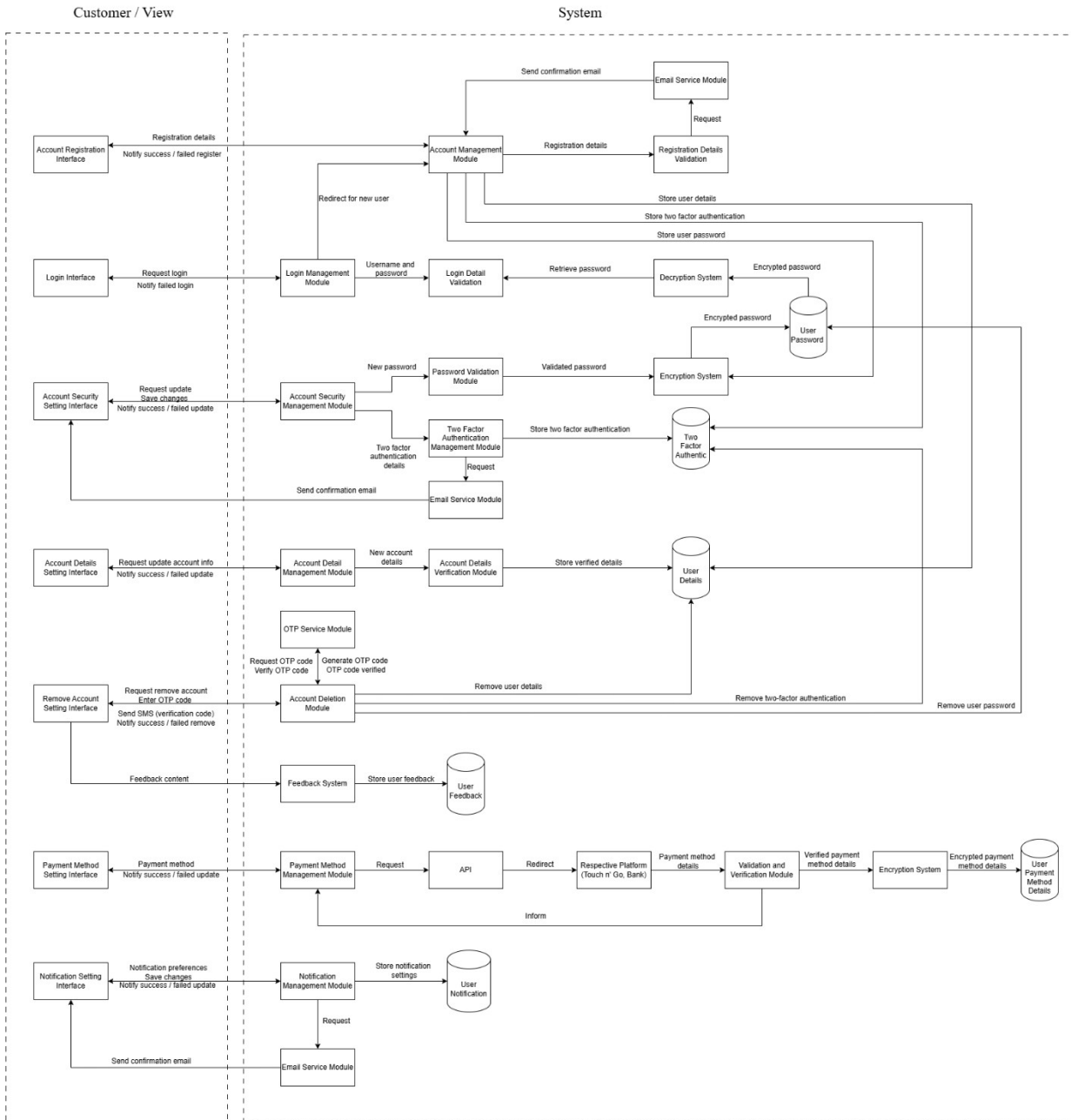


PART I: System Requirement and Design (Individual Task)

Architectures Source: https://drive.google.com/file/d/1DNBt5Wo5--PpN5Dubj0R_MFUTYzZsDxl/view?usp=sharing

1.1 User Account Management System Architecture



Explanation on User Management Module

The user management subsystem provides seven key functionalities: registration of a new user, login of an existing account, modification of users' account information, ensuring user accounts' security, toggling receiving notification via email, deleting user accounts, and

modifying preferred payment methods. The main architecture used here is the Module View Controller architecture. This architecture creates clear separation of concerns between views, modules or controllers, and the database. There are multiple views being used, each tailored for respective functions. Additionally, some of the views use more than one controller or module or vice versa. For example, the removing account view uses both the Feedback System and the Account Deletion Module. The same example can also be used to emphasise loose coupling – the same view uses multiple modules.

When using the system, users are required to log in with their username or email address and a password, given that they had already made an account. Otherwise, users are required to register for an account with their username, email address, password, and phone number. Optionally, the user can insert their gender as well as their date of birth. This information is important to verify if the user is authentic the next time they log in. The additional, optional account information is meant for events that the user may be interested in.

If the email address submitted has already existed, the user will be asked to provide an alternative email address. Once the account details have been submitted, a confirmation email will be sent to the user. Once the user confirms, the user will be inserted into the database and be directed to the login page; if not, the user's account will not be created. Once logged in, the user will be shown the system's homepage. To access the account management features, the user must navigate to the dedicated account management page.

On the account details page, users can view their inserted name, email, phone number, gender, and date of birth. If the user wishes to update their information, they can do so by providing new information to desired fields. When the new changes are saved, information of the user in the database is updated. To the user, the process of updating personal information is quick and easy. This is to ensure that the user gets a seamless and hassle-free experience.

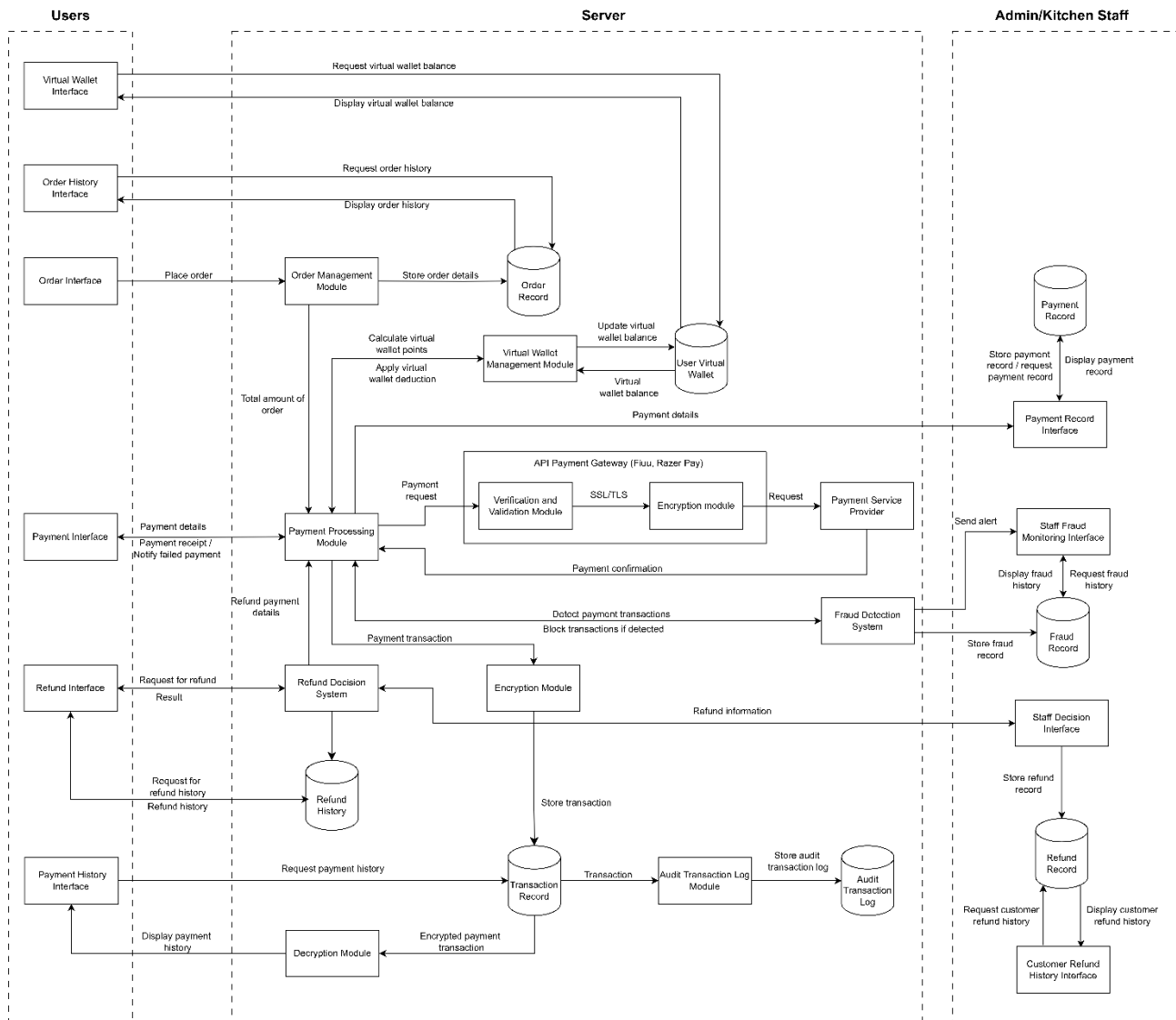
To strengthen accounts' security, an account security page is created to give users an option to enable two-factor authentication or change their passwords. When the two-factor authentication is enabled, the user will insert an alternate email address to receive a confirmation email. Once enabled, every time the user logs in, a verification code will be sent to the user's email, which is needed to complete the login. On the same page, users can change their passwords; they will have to provide their current password and a new password. When the user submits, the inserted current password will be verified against the database; if it exists, the process will shift to validating the new password; if not, the user's request for a password

change is denied. If successful, the new password is saved, and the user is logged out of the session.

If the user wishes to delete their account, they will receive an OTP code sent to their email for confirmation. Three attempts are given to the user to fill in the correct OTP. If the limit is exceeded, the user will be notified that there is an attempt made to delete their account via email. If the OTP is correct, the user will be led to a page for them to provide us with feedback. Once successful, the user information is deleted from the database.

The system supports payment via debit or credit, e-wallet, and cash-on-delivery. If the user wants to use debit or credit, they will insert their card information. If the user wants to use an e-wallet, they would be brought to a page from their selected e-wallet for confirmation. Finally, if the user wishes to use neither, they can opt for the cash on delivery method. Once confirmed, their payment method will be updated in the database and used on their next purchase.

1.2 Payment System Architecture



Explanation on Payment System Module

Based on the payment system architecture diagram, it illustrates the interaction between different components within a food ordering and delivery system. Users are able to use the functionalities provided by the payment system via different interfaces such as virtual wallet, order, payment, refund, order history, payment history, payment record, staff fraud monitoring, staff decision, and customer refund history interfaces.

When a user places an order via the Order interface, it is processed by the Order Management module and stored in the Order Record database. Simultaneously, the total order amount is calculated and passed to the Payment interface. The user also will be redirected to the Payment interface to complete the payment process. For payments, users are allowed to make payment using various methods. For instance, virtual wallet, Touch n' Go, credit card, debit card and so on. If a user selects virtual wallet as their payment method, the Virtual Wallet

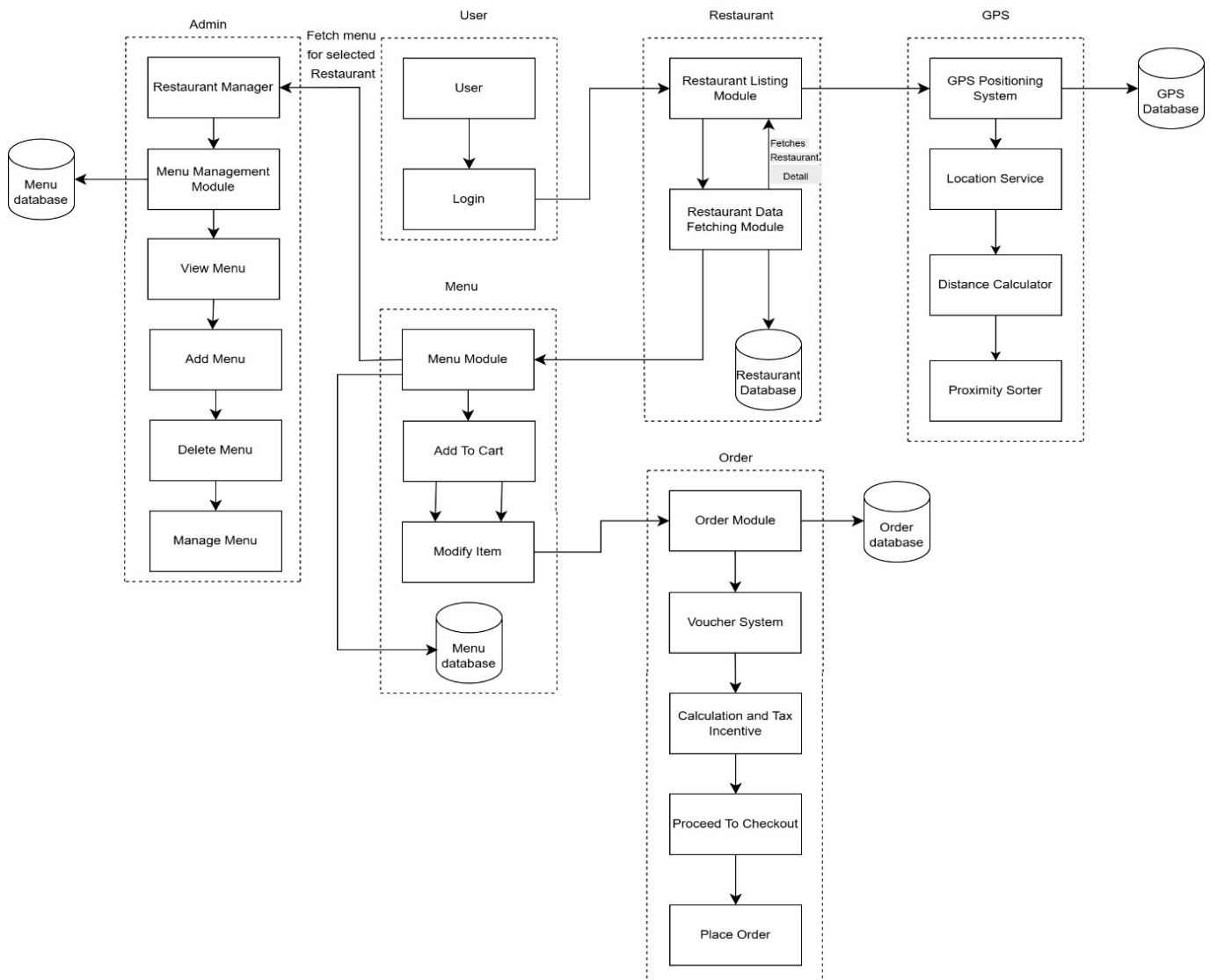
Management module will retrieve the virtual wallet balance from User Virtual Wallet database and calculate the remaining amount that needs to be paid. Initially, the user is required to enter payment details through the Payment interface. These details are being processed by the Payment Processing module. It will interact with an external Payment Gateway API.

Under the API Gateway, the transaction data passes through a Verification and Validation module to ensure its authenticity and accuracy. After that, the data is encrypted by the Encryption module. To ensure a secure transaction, the entire data transmission is protected using SSL/TLS protocols. Once the data has been verified and encrypted, a payment request is sent to the respective Payment Service Provider. After the transaction is completed, the result including the payment receipt and confirmation is displayed to the user. When a failed transaction occurs, the system will notify the user and prompt them to retry the payment. All transaction details are securely encrypted and stored in the Transaction Record database. These records are also logged by the Audit Transaction Log module to ensure traceability and accountability within the payment system. Moreover, once the payment is successful, the virtual wallet points earned will be calculated and updated in the User Virtual Wallet database by the Virtual Wallet Management module. Simultaneously, the system notifies the respective restaurant management by forwarding payment details to the Payment Record interface and stores them in the Payment Record database for future reference.

If a user initiates a refund request through the Refund interface, the request is evaluated by the Refund Decision System. Once the request is approved, the refund result is routed back to the user. Additionally, the updated transaction data will then be encrypted and updated in the Transaction Record database and Audit Transaction Log database. The system allows users to view their refund history, payment history, order history, and the balance of their virtual wallet through their respective interfaces.

To maintain system integrity, a Fraud Detection System monitors all transactions for suspicious activity. It has the capability to detect and block fraudulent transactions. If a suspicious transaction is detected, the system will alert the Staff Fraud Monitoring interface and record fraud details in the Fraud Record database. Staff can review and respond to these alerts through the Staff Decision Interface. In addition, if staff receive any refund request from the Refund Decision System, they can evaluate the request using the same interface. Staff can also review customer refund histories via the Customer Refund History interface. Meanwhile, kitchen staff can access payment records and manage refunds through the Payment Record interface.

1.3 Menu Management System Architecture



Explanation of Menu Management System

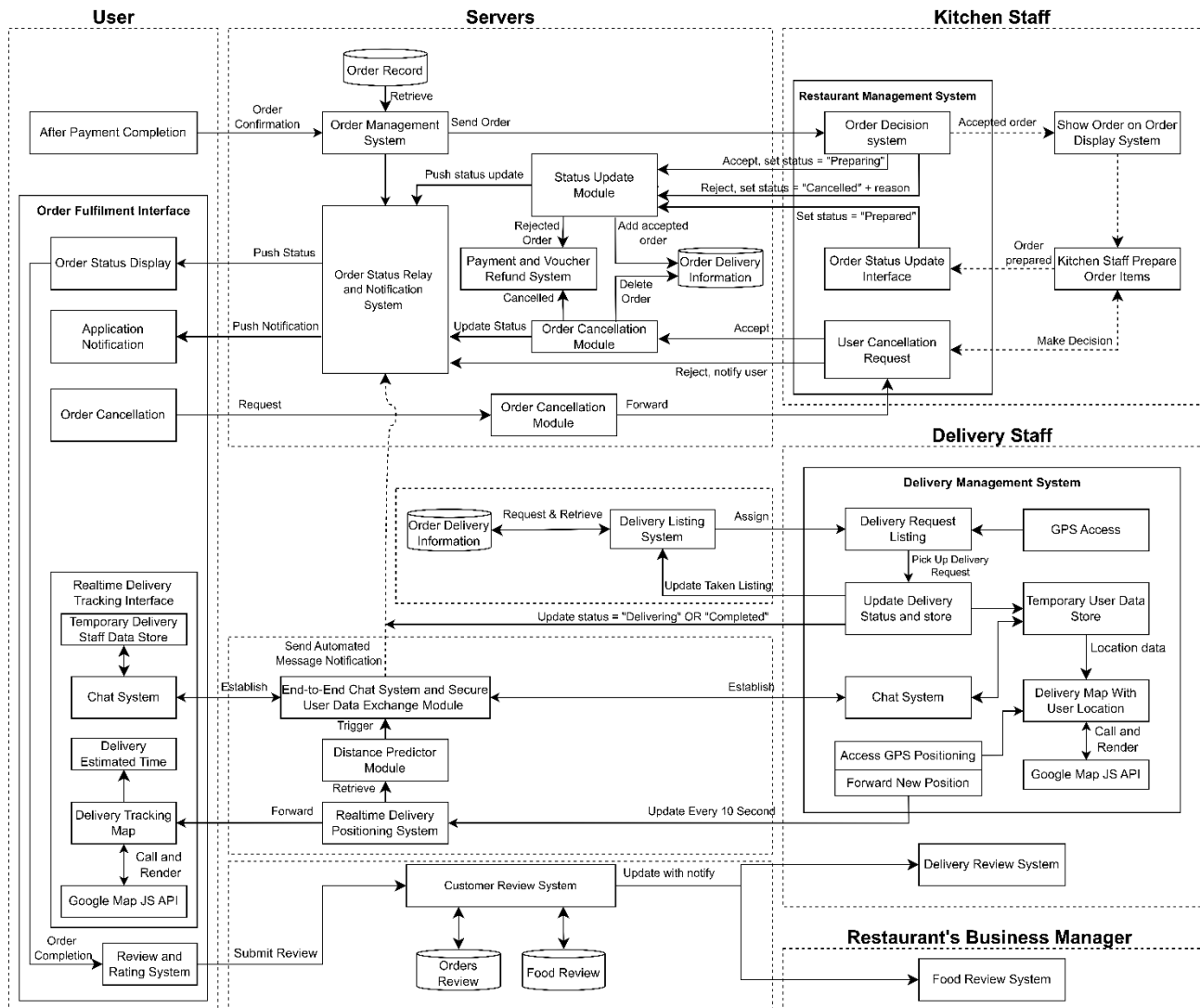
The food ordering system depicted in the diagram integrates multiple components, ensuring a seamless experience for users, restaurant managers, and administrators. The system begins with user authentication, where individuals log in to access restaurant listings and menu options. The Restaurant Listing Module displays available dining options, and the Restaurant Data Fetching Module retrieves specific restaurant details from the Restaurant Database. Once users select a restaurant, they interact with the Menu Module, which displays available food items fetched from the Menu Database. Users can then add items to their cart, customize their orders through the Modify Item function, and proceed to check out.

On the administrative side, restaurant managers handle menu updates via the Menu Management Module, which allows them to view, add, delete, and manage menu items stored

in the Menu Database. This ensures that users always have access to the most up-to-date menu options. Once an order is placed, the Order Module manages the process, applying potential discounts through the Voucher System and calculating the final cost, including taxes, via the Calculation and Tax Incentive Module. Users then confirm their order in the Proceed to Checkout section, which finalizes the transaction in the Order Database.

Additionally, the system incorporates GPS-based services to enhance the user experience. The GPS Positioning System fetches the user's location from the GPS Database and utilizes the Location Service Module to calculate distances and sort restaurants based on proximity. This feature helps users find nearby restaurants quickly and efficiently. Overall, the system ensures a streamlined process from menu browsing and order customization to checkout and restaurant recommendations, making food ordering convenient and efficient for both customers and restaurant administrators.

1.4 Order Fulfilment System Architecture



Explanation on Order Delivery Fulfilment System

After payment is successfully made, the Order Fulfilment System is deployed to fulfil the user order, by facilitating staff productivity in their daily routines to enhance efficiency in delivering the foods to customer and improves business growth. This system ensures that customer orders are fulfilled seamlessly by managing the flow of order confirmation, preparation, delivery and review between the customer-side system and business-side system, ensuring a good user experience and delivering strong business values for restaurants.

Firstly, after the user makes a payment, the system forwards the order to the restaurant's kitchen Order Decision System for review. This system allows staff to either accept or reject the order based on the restaurant's current situation and the availability of resources to prepare the ordered food. If the order is accepted, the system notifies the customer about the order's

preparation and simultaneously places a delivery request to assign a delivery person, which can save overhead time to search for delivery staff after food is prepared. Else, if the order is rejected, the system manages and notifies the user about the order cancellation, providing reasons from the restaurant staff. The system also processes refunds for payments and vouchers, ensuring customers are informed about the situation and avoiding any potential misunderstandings.

After order acceptance, the user of the system can still request order cancellation from the restaurant staff, where the staff can decide based on their order progress. Therefore, if staff allow the cancellation, the system will manage the cancellation step including refunding payment, voucher and cancelling delivery request for that order. Else, it will notify the user about cancellation request is rejected by restaurant and the order process will continue.

Following, the Delivery Management System plays an important role in fulfilling the Order Fulfilment System. Delivery staff are notified of new order delivery requests and can pick up only one request from the listing when they have no ongoing deliveries. Once a request is picked up, it is marked as taken in the system. The assigned delivery staff will receive the necessary user information related to the order, which can be used to establish an end-to-end chat room with the customer. This feature ensures that the delivery staff and the customer can communicate directly in case any problems happened during the delivery process.

In addition, the Order Fulfilment System features a Realtime Delivery Tracking System that polls the delivery staff's GPS data to a server every 10 seconds. This data is then forwarded to the customer's Realtime Delivery Tracking System, allowing the customer to view the staff's location on API integrated Google Map. This feature enables users to track their order's delivery progress along the route and prepare to collect. Furthermore, the tracking system provides users with an estimated delivery arrival time and automatically sends notifications and chat messages when the delivery is near, ensuring user will get their order around the given time.

Lastly, after order fulfilment, the Review and Rating System is activated, this allows customers to share their experience and provide feedback on both the delivery and food. This system can guide the restaurant, business owners, and delivery staff identify areas for improvement such as food quality, packaging and delivery speed to maximize their customer confidence and lead to profit in long term. Therefore, by utilizing customer reviews system, the business can maintain and improve its services through valuable insights and reflection, bringing sustainable growth to business.

PART II: SEP Process and Questions

2.1 Q1: Requirement Analysis and Design Principles (Group Task)

2.1.1 Key Functionality Requirements

User Account Management System

1. Users must create an account to use the online Food Ordering and Delivery System.
2. The system should verify if the email has been used during account registration.
3. The system should verify if the password inserted meets the requirements.
4. The system should send an email for account creation confirmation to the user.
5. The system should validate login details.
6. The system should display login status.
7. The system should display registration status to the user after account creation.
8. The system should save new users into the user database.
9. Users should be able to view their account information.
10. Users should be able to make modifications to their account information.
11. Users should be able to toggle two factor authentication.
12. The system should send an email for confirmation based on the email inserted during the setup of two factor authentication.
13. Users should be able to change their password.
14. The system should check if users' current passwords match with the password that the users inserted.
15. The system should check if the users' new passwords meet the requirements.
16. The system should update the users' accounts' password in the database.
17. Users should be able to toggle receiving updates via emails.
18. Users should be able to permanently delete their accounts.
19. The system should send an OTP to users to ensure that the requests made are legitimate.
20. The system should display account status to the users.
21. The system should permanently delete users' accounts if successful.
22. Users should be able to change their mode of payment.
23. The system should update the users' modes of payment in the user database.

Food Catalogue and Ordering System

24. Users can view the list of available restaurants based on their operating hours.
25. Users can browse the catalogue of menu items for each restaurant.
26. The menu listing should always reflect to the items latest stock and item availability.
27. Users can apply filters.

- 28. Users can browse the food catalogue.
- 29. Users can choose their food from the food catalogue.
- 30. Users can filter and search for food.
- 31. Users can add items to a cart.
- 32. Users can customize food items.
- 33. Users can modify items in the cart.
- 34. Users can review their order summary before final checkout.
- 35. System must calculate and display the total cost including taxes.
- 36. System shall allow adding new menu items and updating existing ones.
- 37. System must support order modification before preparation begins
- 38. Restaurant managers can view and manage incoming orders.
- 39. Restaurant owners can respond to customer reviews.

Payment System

- 40. Users should be able to view the food menu via an interface.
- 41. Users should be able to make payment after placing an order.
- 42. Users should be able to pay the payment with multiple payment method.
- 43. Users should be able to view back their previous payment records and order records.
- 44. Users should be able to view back their virtual wallet balance.
- 45. The system should perform validation and verification of payment details and transaction processing.
- 46. The system should generate payment confirmation and payment receipt to the users after successful payment.
- 47. The system should notify the restaurant admin about the order payment after successful payment.
- 48. The system should inform users if the payment process failed.
- 49. The system should calculate the virtual wallet points after successful payment and update to the database.
- 50. The system should be able to detect suspicious transactions and alert the system administrator.
- 51. The system should be able to block payment transactions after detecting fraud.
- 52. Users should be able to retry payment after a failed payment attempt.
- 53. Users should be able to make payment with the virtual wallet.
- 54. Users should be able to make request for refund after a payment has been made.

- 55. Users should be able to get back the original payment amount after their refund request has been successfully approved.
- 56. Admins are allowed to monitor and maintain all payment transactions.
- 57. Admins should be able to view customers' refund history.
- 58. Admins should be able to receive alerts about fraud detection.
- 59. Admins should be able to view the history of fraudulent transactions.

Order Fulfilment System

- 60. System should send the user's order to the Kitchen Staff after payment completion.
- 61. Kitchen staff can decide to accept or reject order based on their situation.
- 62. User should be able to track their order status via order fulfilment interface and get latest notification.
- 63. The system should add the user order's delivery request to allow nearby delivery staff to pick up the request.
- 64. User should be able to cancel their order due to change of mind or voucher modification, while the cancellation request is only decided by kitchen staff or restaurant manager.
- 65. User should get back their amount paid and voucher after successful order cancellation.
- 66. The system should remove cancelled order's delivery request.
- 67. The system should ensure the delivery staff to permit GPS accessing to search for nearby delivery request and used for the delivery tracking.
- 68. Delivery staff can view all the available delivery request nearby.
- 69. Delivery staff can view and accept one delivery request per time.
- 70. Delivery staff can view a delivery map with delivery destination.
- 71. The system should update the delivery request listing after delivery staff pick up a delivery request.
- 72. User should be able to track their order on the map with estimated arrival time
- 73. User can chat with his/her delivery staff to give instructions and vice versa.
- 74. The system must automatically notify users with order arrival reminders when the driver is approaching their delivery address.
- 75. Delivery staff can view their delivery rating and comment by the user.
- 76. Restaurant manager can view their restaurant's food rating by the user.
- 77. User can provide delivery and food feedback with rating after order fulfilment.

2.1.2 Non-Key Functionality Requirements

1. The system should have a strong firewall in place to prevent data breaches.
2. The system should send confirmation email to the user as soon as possible.
3. The system should fetch and display users' account information in less than 2 seconds.
4. The system should send OTP to the user as soon as possible.
5. The system should have a secure and encrypted payment interface.
6. The system should work consistently and responsive throughout the restaurant business hours.
7. The system should support high volumes of simultaneous order requests without performance degradation.
8. System should implement automatic recovery mechanisms for common failure scenarios.
9. Order interface must be intuitive with clear navigation paths.
10. Order status should be clearly communicated to users.
11. Order history should be easily searchable and filterable.
12. Order processing logic should be modular and well-documented.
13. Database design should optimize for high-volume order processing
14. The payment system should be available 24/7 without downtime.
15. The payment interfaces should be user-friendly and accessible for people with disabilities.
16. The payment interfaces should support multiple languages.
17. The payment system should handle 10,000 concurrent payment requests.
18. The payment system should ensure that payment transactions are accurate and without duplication.
19. Payment processing should be complied with security standards like PCI-DSS.
20. SSL/TLS encryption must be used for payment transaction in the payment system
21. Payment processing should be completed within 5 seconds.
22. The virtual balance, payment history and order history should be display within 2 seconds to the users.
23. All payment transactions must have detailed logs for audit and monitoring purposes.
24. All payment transactions must be encrypted in database.
25. Order processing where the kitchen staff should receive their order within 5 second.
26. The system should accurately track the delivery staff location every 10 second polling.

27. The real time map tracking should handle up to 10,000 concurrent users without noticeable delays in response.
28. The system should always use End-to-End encryption to protect all users (User and Delivery Staff) data and chat across the internet.
29. The system should support the increasing of users during peak hours like lunchtime (1pm-2pm), dinnertime (6-8pm), weekend and public holidays.
30. The system interface and design should follow 3-click-rules to ensure user could find what they need without hassle.
31. The system can be updated with minimal downtime no more than 2 hours or during early morning hours (12.00am – 5.00am) where there is no operating restaurant that time.
32. The system should support Malay and English Language only.
33. The system is designed in a way that allow future integration of new module or system easily with less modification.
34. The important module like menu listing, order processing systema and real-time delivery tracking should be scaled easily under high load.
35. The system must ensure that critical systems are isolated and capable of functioning during system failures. This ensures that the delivery tracking service remains operational, ensuring order deliveries are fulfilled seamlessly and fulfilling the system goals under failure.

How would design principles such as modularity, scalability, maintainability, and security guide your architectural solution?

2.1.3 Modularity

Modularity is a design principle that breaks down complex components into smaller, manageable pieces called modules, simplifying complex software development. Each module has their own responsibility and can be used individually or combined with other modules to create a larger function. The Food Ordering and Food Delivery system is broken down into multiple smaller, manageable components – user account management system, payment system, menu management system, and order fulfilment system. Moreover, each of the components is broken down further into smaller functions. This makes the development of the system easier since each component can be developed one at a time and the functions can be developed in parallel. Additionally, one of the key characteristics of modular design principle is loose coupling. This means that modules or functions are connected only via their outer layers, eliminating the need to know about how each other work. For example, one module that is responsible for displaying modification status can be reused for different modules, eliminating the need to constantly rewrite the code from scratch, speeding up overall development time.

Moreover, with modularity, changes can be done to one component independently, improving maintainability; since each component has its own distinct responsibility and can operate independently, changes made within a component remains confined. For instance, developers can modify the menu management component without affecting the user account management component. If a component must be replaced, it can also be done so with minor changes, given that the interface of the component matches existing components. This reflects loose coupling; since modules are only connected via their outer layers, as long as the interface remains consistent, the rest of the system will not be affected. This also make debugging easier, where the developers can pinpoint the source of the issues quickly, preventing them from wasting time checking unrelated components.

Finally, with modularity design principle, teams can work on different components in parallel, making completing a project much faster. This works because teams working on different components do not need to have an in-depth understanding of the internal workings of other components; they only have to know the interface through which the components will interact with each other, reducing communication overhead and potential roadblocks.

2.1.4 Scalability

Scalability is a design principle that enable a system to handle increasing users loads in a system to prevent service availability to some or all user due to unprecedented loads spike during the system operations. As for online Food Ordering and Delivery System just like Food Panda and Grab Food, the system needs to carter the sudden spike of user using the system to order food during peak hours like lunchtime and dinnertime. During this period, the system's server resource utilization will be higher than usual, and the system must handle it gracefully by scaling resource limit. Therefore, the response time of the system is minimized to maintain user experience.

To maintain high availability of system's core features, such as restaurant and food listing, payment processing, and order delivery fulfilment system during peak hours, the system is distributed into two or more servers. These techniques allow a system to be scalable by load balancing and managing the incoming and outgoing traffic, therefore this minimizes the response time of the system. Furthermore, load balance helps the system to dynamically adapt to sudden fluctuations in resource utilization of each server, preventing the service disruptions and prevent service denial.

Scalability becomes more accessible with cloud computing service providers. For instance, Amazon and Alibaba provides Elastic Computing Service (ECS), which can host the Food Ordering and Delivery System while providing seamless, flexible and zero-downtime resource scaling. These ECS offers automatic horizontal scaling, which creates new instance of same hosted system to support the workload distribution by load balancing. When the utilization is low, the instance of server is automatic removed to save cost. Therefore, this is an effective way to maintain the system at optimal performance while minimizing the cost based on resource utilization.

To support the scalability, each important component like restaurant and search, payment processing and delivery tracking should be modularized into a few isolated services. This approach allows each service to be scaled vertically by increasing computational resources (CPU or GPU) or horizontally by deploying multiple instances. For example, real-time delivery tracking can operate efficiently and undisturbed within an isolated processes for each user, ensuring seamless scalability and performance under high traffic periods. This separation ensures that the tracking system continues to function, fulfilling user orders even during downtime of the Food Ordering and Delivery System.

2.1.5 Maintainability

Maintainability stands as one of the most critical design principles in software architecture, fundamentally shaping how systems evolve and sustain value throughout their lifecycle. When architects prioritize maintainability, they create systems that can be efficiently updated, enhanced, and operated over extended periods, often far beyond initial expectations. The essence of maintainable architecture lies in its ability to accommodate change with minimal friction as systems inevitably evolve in response to shifting business requirements, technological advancements, and emerging user needs.

Modularity forms the cornerstone of maintainable architecture by decomposing systems into discrete, loosely coupled components with well-defined interfaces, creating natural boundaries that contain complexity. This approach yields benefits including reduced risk from localized changes, possibility for parallel development, and more targeted testing. Closely related is the principle of separation of concerns, which carefully distinguishes between different aspects of the system presentation logic, business rules, data access, infrastructure concerns to preventing them from becoming entangled.

Maintainable architectures also exhibit consistency in patterns, conventions, and technologies, requiring architectural governance to establish and enforce standards while balancing standardization with flexibility. Even the most elegantly designed system becomes unmaintainable when knowledge about its structure and behaviour is lost, making effective documentation crucial for capturing architectural decisions, system constraints, and design rationales. Architecture Decision Records (ADRs) have emerged as a particularly valuable tool in this regard. Systems must also be inherently testable, with clear component boundaries, dependency injection capabilities, pure functions, and observable behavior. Since no architecture remains optimal indefinitely, maintainable systems provide mechanisms for managing technical debt through refactoring pathways that allow incremental improvements.

Maintainability extends beyond code to encompass operational aspects through appropriate monitoring, logging, alerting, automated deployment processes, and site reliability engineering practices. Ultimately, architectural maintainability emerges from consistent application of principles that acknowledge the inevitability of change, with its true measure being how effectively a system can adapt to tomorrow's challenges while preserving investments already made.

2.1.6 Security

Security is one of the design principles that ensures sensitive user data - payment transaction details, account information, and personal details - stay protected and kept from cyber threats. A robust security framework is a pertinent requirement for an online Food Ordering and Delivery System, because any security breach can compromise customers' online security, resulting in an erosion of customers' trust and ultimately results in financial losses for the company. Therefore, there are three fundamental security principles such as confidentiality, integrity, and availability that should be applied to establish a strong cyber security shield.

First, confidentiality is important to ensure that sensitive user data such as usernames, passwords, and personal details remain private and cannot be accessed without proper authorisation. This is achievable through the implementation of encryption and secure transmission protocols to stop unauthorized individuals from accessing private internal data. For instance, payment data should be encrypted with an encryption algorithm, such as AES-256, before transmission and be transmitted over a TLS. With end-to-end security, sensitive payment details can be prevented from being exposed to malicious, unauthorised users while they travel across the network. Additionally, even if the sensitive data is intercepted during transmission, the encrypted data will be difficult to decrypt without knowing the encryption process.

Furthermore, integrity is to effectively ensure that the sensitive data is complete and has not been altered by unauthorized users. For example, to maintain integrity, immutable audit logs and real-time fraud monitoring should be implemented in payment system. In immutable audit logs, every transaction will be permanently recorded, capturing every change to the transactions' history, allowing admin to trace the source of any alterations. In addition, real-time fraud monitoring such as Intrusion Detection Systems (IDS) can help in detecting and flag any suspicious activities immediately and inform the administrators, to prevent data manipulation.

Moreover, availability ensures that authorized users can access services when needed. To maintain availability, regular security patching is essential for the payment system to prevent any vulnerability that can be exploited. Simultaneously, it helps to reduce the risk of downtime by addressing issues before any system failures can occur due to said issue. In addition, implementing mechanisms such as traffic filters and firewalls ensures that the system remains operational in case of unexpected disruptions.

In conclusion, proper security measures ensure smooth system operation with minimal downtime, fulfilling the security design principle.

2.2 Q2: Pattern Selection and Justification (Group)

In designing the 4 system architectures solution, we had implemented several key design patterns, with the most notable design pattern like Observer, Abstract Factory, and Proxy design patterns. These patterns are the base of Food Ordering and Delivery System, particularly the transaction management playing a role in maintaining the modularity, scalability, maintainability and security. Therefore, this section will discuss on how the design patterns had supported our architecture design in achieving the system requirements with explanation, and how the patterns contribute to solving different design challenges within the systems modules.

The observer pattern is a behavioral pattern that Food Ordering and Delivery System. This pattern allows dynamic update across different modules or components from a server like real-time menu refreshes, payment refund status update, and the user's order delivery status and tracking system. These features observe the changes in the server and perform synchronize seamlessly to retrieve the latest information, ensuring effective integration between each module or components.

The Order Status Relay and Notification Module in the delivery fulfillment system utilizes the Observer Pattern. This ensures that any updates to the order or delivery status, whether made by the user, kitchen staff, or delivery staff, are instantly observed by to all relevant participant of the system. Each participant in the system will receive real-time notifications and their session are immediately updated with the latest status on the server, this creates a seamless communication and transparency throughout the order delivery fulfilment process and allowing users to track their orders. Additionally, the delivery tracking system uses the pattern, with delivery staff updating their GPS positions while users observe their location updates on the map.

Moreover, the menu listing system and delivery requests display system adopts the Observer pattern to provide real-time updates for the menu and delivery requests listing respectively. Increasing update demands will burden the server resources and the network bandwidth, leading costly maintenance and difficulties to scale the system. To prevent this, the architecture is designed to allow server to push any update directly to subscriber interface, following the Observer pattern. This approach converts the complex 2-way communication (Request-Response) into 1-way (Push) system, therefore preventing the client requests from flooding the server, but instead, the server will push new update and directly delivered to subscriber interface, offering efficiency, reduced maintenance cost and reduced internet traffic.

The Food Ordering and Delivery System development needs precise choice of design patterns which provided modularity along with scalability and maintainability attributes. The Abstract Factory pattern proved essential among all patterns implemented to manage the diverse aspects of our system design. The system benefited from this design pattern to handle payments through distinct platforms because it supported various payment approaches while keeping all interfaces standardized. Through the Abstract Factory pattern, we established a framework to generate objects from related families without selecting concrete implementations thus our system could interact with payment processors using one coherent interface which included credit card, e-wallet and cash-on-delivery options. New payment methods could be added to the system through this approach while maintaining existing code functions because of the software design principle Open/Closed.

Within our menu management subsystem, the Abstract Factory pattern enabled our system to manage different restaurant requirements for distinct menu structures and item categories and customization options. The pattern enabled our application to create abstract menus using interface definitions but let concrete factory systems decide the actual implementations matching restaurant specifications. The system improved its efficiency to accept new restaurant types through its standard approach which preserved uniformity in handling menu items across all restaurant divisions.

Through the Abstract Factory pattern the Order Fulfillment subsystem prepared customized notification methods for different types of system users. The notification factories produced role-specific messages for customers and delivery staff and staff members through separate facilities which shared a shared notification framework.

Our system obtained a robust architectural foundation through the implementation of Abstract Factory pattern together with the Observer pattern along with Strategy pattern and Singleton pattern. Combinations of Abstract Factory pattern with Observer and Strategy and Singleton patterns created a solid architectural base through their ability to control related object creation and notification management between components and algorithm selection and shared resource access. System flexibility and maintainability and extensibility developed through the precise use of these patterns resulted in Food Ordering and Delivery System which meets current business demands and technological progress.

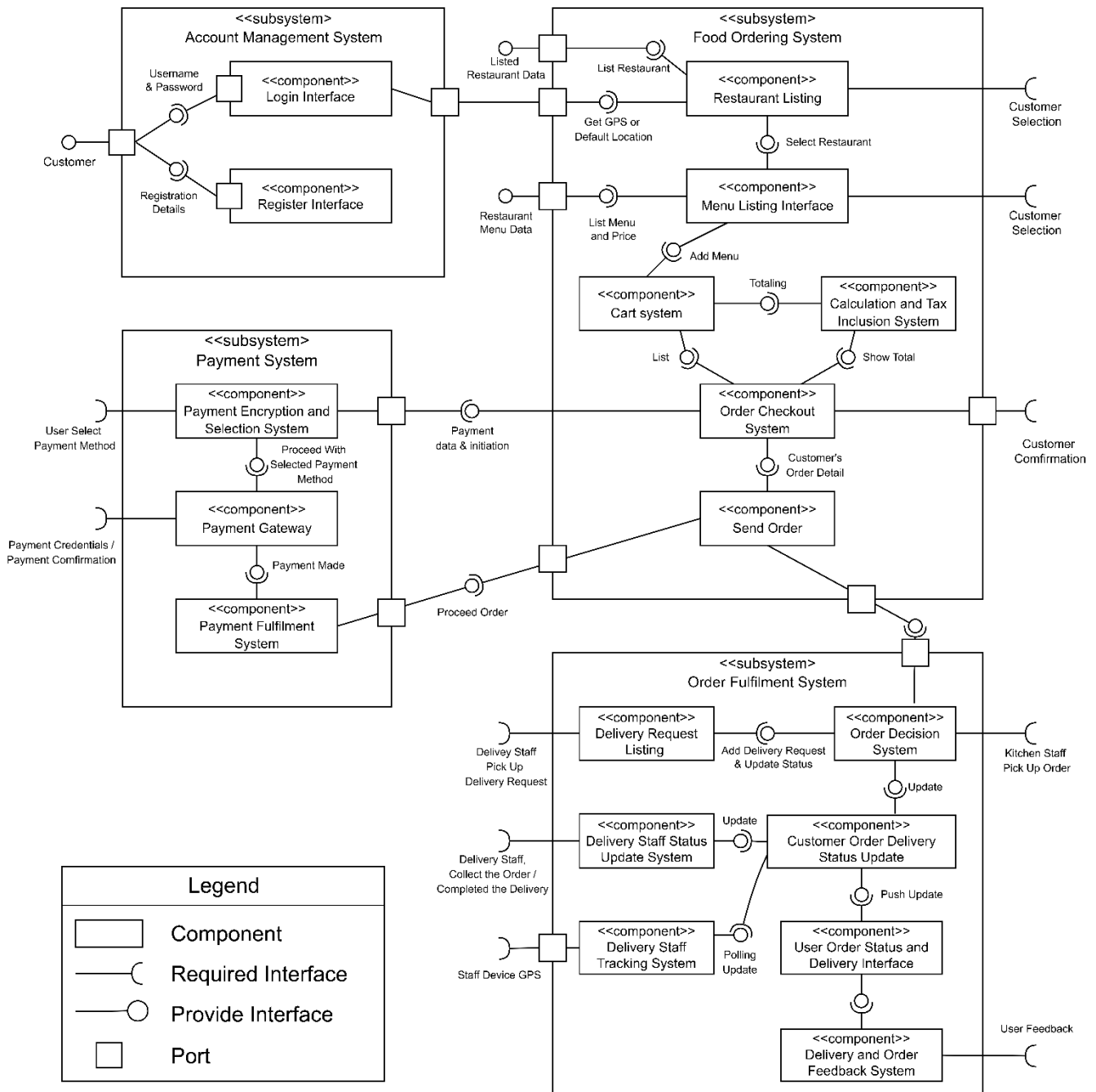
Proxy pattern is a structural design pattern that used in the food ordering and delivery system. It provides a placeholder or intermediary that control the access to internal components; instead of letting the clients directly interact with the real components, such as payment gateway, user sensitive information and so on, it allows clients communicate through a proxy, protecting the system from malicious activities, and user errors. This proxy acts an intermediate layer that manage user access and perform additional operations such as authentication, validation and logging before sending requests to the desired components.

For example, in the payment system module, the proxy pattern is applied to secure the interactions between clients and external payment gateways. When a user initiates a payment, the proxy verifies and validates the payment details before forwarding them to the external payment gateway. This prevents malicious or invalid requests from reaching the gateway, inadvertently improving the security of the overall system from attacks.

In addition, the proxy pattern can used to facilitate logging user activities and system operations. This is possible because of the location of the proxy - between clients and components, allowing it to capture and record every detail of operations, including transaction attempts, modification attempts and error occurrences. This logging process helps system administrators to monitor system usage – high or low traffic - and identify abnormal behaviours – extreme spikes or dips in activities - allowing them to take actions quickly and ensuring an honest auditing process. By monitoring the system usage, the team can prepare any issues that may occur during busy hours. At the same time, the logs can serve as valuable data for intrusion detection systems (IDS) to learn suspicious patterns and enable them to recognize potential threats more quickly and accurate, allowing them to take measures that prevent the predicted malicious activities from happening.

Finally, the proxy pattern is also used to control access to internal, sensitive components. For instance, each user can only view their own order history, transaction history and details, virtual wallet balance, and other sensitive personal data. The proxy checks the user's account and access rights to prevent users from accessing account details of other users. The same applies when users wish to change their information; only authorised users can modify their own personal information. With the proxy, data privacy and security are enhanced by only permitting the only authenticated and authorised users can interact with the sensitive parts or information of components in the system.

2.3 Q3: Component and Architecture Design (Group Task)



In the Account Subsystem, the user will first login or register an account. In Login Interface, the user must provide the correct username and password. If the details are incorrect, the user will not be able to log in. The Register Interface component will let the user create an account and insert the user into the user database.

Once the user has logged into the system, the user can start ordering their food with the Food Ordering Subsystem. Before the user can start viewing restaurants, the system will first get the user's location, either from GPS or from the default location that the user has set. After that, based on the user's current location, the system will list the restaurants from the database

in an intuitive interface for the user to easily follow. This is done with the Restaurant Listing component. When the user selects a restaurant, the menu from that restaurant will be fetched and shown to the user via the Menu Listing Interface component. The user can then start ordering the item that they want. The user will use the Cart System component to save and keep track of the food that the user wants. Once the user has finished ordering their food, the total amount to be paid will be calculated by the Calculation and Tax Inclusion System component. This component will first receive the original sum of the prices of all the saved items from the Cart System component. Once the new total has been calculated, the new total and the list of items will be passed into Order Checkout System component to show to the customer.

When the customer confirms their order, Payment Subsystem will be called. In this subsystem, the Payment Encryption and Selection System component will be called to ensure that the user's payment information is encrypted and prevented from exposure from outsiders. After that, the payment will be proceeded with the Payment Gateway component to verify payment credentials. After that, the Payment Fulfilment System component will be called to ensure that the payment goes through and confirms the order. Once the order has been confirmed, the order will be sent to the chosen restaurant to be prepared.

The system now moves to the Order and Delivery Fulfilment Subsystem which will track the status of the order. The components in this subsystem interact with each other in both directions. The Order Decision System component requires the restaurant staff to confirm, reject, or call the customer to modify the order. If the order is confirmed, Delivery Request Listing component will request surrounding available delivery staff to pick up the delivery request. If there is a willing delivery staff, Delivery Staff Status Update System component will push an update - information of the delivery staff - to the Customer Order Delivery Status Update component. At the same time, the location of the delivery staff will be tracked with Delivery Staff Tracking System component. The Customer Order Delivery Status Update component's role is to manage the updates before showing the user. The User Order Status and Delivery Interface component will control how the customer gets the status of the order itself. Finally, when the user's order has been delivered, Delivery and Order Feedback System component is called to request user to provide feedback on their order.

2.4 Q4: Reflection on Design Choices (Individual Task)

2.4.1

To design my architectural solution, I have investigated multiple design patterns or techniques as well as architectural design to help guide me to create a robust yet efficient solution. The architectural design used in the user management system is Model-View-Controller or MVC architectural design. With this architectural design, a clear separation of concern in the components is created in the subsystem; the user interfaces, business logic, and the data are separated. This architecture improves modularity and reusability; each view or interface is independent and multiple views or interfaces can use the same modules or controllers. As a result, this architecture allows me to replace interfaces, controllers, and modules without worrying about affecting other components, improving reliability and maintainability. For instance, I can remake and replace the Login Interface without needing to replace the Login Management Module. This architecture also emphasises loose coupling, where each component is general enough to be reusable for other situations and uses. For example, the Encryption System is used for both account security function and account registration function.

A design pattern that is paired with MVC architectural design is the modular design pattern. The modular design pattern involves breaking down a complex software into smaller, more manageable components. For example, the account registration function involves several subfunctions or subsystems such as Validate Registration Details, Email Service Module, and Encryption System. This means that each function can be developed, maintained, and reused independently. Additionally, each of the subsystems can be broken down into more subsystems. Modular design lets me improve maintainability, scalability, testability, team collaboration, and reduced complexity when done properly and within specific boundaries.

I learnt that most software is written based on multiple architectural design and design patterns. Often, architectural design patterns are paired with design patterns to take advantage of synergies or to complement the weaknesses of design patterns or architectural design patterns with strengths of another. For example, modular design pattern pairs well with MVC because both emphasise the importance of separation of concern and loose coupling, making the system more maintainable. Additionally, it also makes the system easier to be developed because each function is broken down into smaller subfunctions, allowing developers to focus on one function at a time.

2.4.2

Pattern-based techniques have significantly influenced my architectural decisions in this assignment. They provide a structured approach to organize the system components. In designing the payment system architecture, the Model-View-Controller (MVC) pattern was applied to separate the presentation layer from the business logic and data management components. This separation not only enhanced code maintainability but also improved the system's scalability and clarity, especially there are multiple views in the architecture. These views cater to various user roles such as customers, kitchen managers, and system administrators. By implementing the MVC pattern, the user interface for each role can be modified or adjusted independently without affecting the underlying data or business logic. Therefore, the MVC pattern provides the flexibility needed for the system to evolve or adapt to future requirements.

By using design patterns in this assignment, I learned that they help me simplify complex problems by providing proven and reusable solutions. For example, the Observer Pattern was utilized to handle notifications within the system. It played a crucial role in notifying staff members about potential fraud alerts in real time. Since design patterns promote code reusability, the same structure can be reused in different contexts within the system. For instance, a similar implementation of the Observer pattern can also be used to alert users about refund statuses or payment confirmations without making any major modifications to the core logic. Therefore, this demonstrates that design patterns not only can reduce the duplicated logic but also make the system easier to extend and maintain.

In conclusion, pattern-based techniques are essential approaches in software development, as they provide structured and reusable solutions that enhance system scalability and maintainability. Each design pattern and architectural pattern has its own advantages and disadvantages, so it is important to apply them appropriately based on the system's requirements. After completing this assignment, I learned that design patterns and architectural patterns are not just theoretical concepts as they can be used as solutions to enhance the robustness and clarity of the system. By using them, I was able to organize and structure the system more effectively.

2.4.3

The system architectural decisions received major influence from pattern-based techniques because these standardized approaches solved recurring design problems effectively. In the Menu Management and Order Modules the Model-View-Controller (MVC) pattern maintained separate components between user interface parts (View) and business operations (Controller) and database interactions (Model). The database separation mechanism enabled automatic Menu Database changes to reach user interfaces without affecting control structures while supporting dietary filter implementation. The Singleton pattern delivered essential consistency to GPS Positioning System and Location Database operations by establishing a single instance for location data to function between Proximity Sorter and Distance Calculator modules. The centralized system reduced additional API demands while decreasing resource demand. Minor discounts within Voucher System achieved better flexibility through the Strategy pattern because it allowed interchangeable discount strategies to encapsulate algorithms such as percentage offers, or fixed amount deals without affecting the core Order Module.

Several fundamental conclusions formed because of the development process. The usage of MVC together with Strategy patterns efficiently solved complex issues through proven solutions which improved development speed while enhancing maintainability. The essential task became how to achieve this balance. The consistent use of Singleton proved essential for location services however repeating its application too frequently introduced problems with global state which led to the need for specific context evaluation. Implementing standardized patterns like Strategy lead to better collaboration by creating a common language which lowered misunderstandings during program execution. Through the Observer pattern which underlays order status notifications the system gained advantages related to scalability. The order database remains separate from user interfaces so the system can add new notification channels like SMS or email without any complications in the future.

Pattern-based strategies exposed how proper design intentions create essential value in the development process. The application of patterns should be handled thoughtfully as tools rather than fixed requirements, so they reinforce adaptive systems in complex Business Information Systems.

2.4.4

Before implementing architectural design, pattern-based techniques have played a significant role in my architectural decisions. The most notable architectural pattern in the Order Delivery Fulfilment System is microservice architecture. This pattern ensures that each system and its components remain independent and loosely coupled, allowing for greater scalability and fault tolerance gained from microservice architecture benefits. As a result, if any individual component fails, it does not lead to a complete system failure. Instead, critical functions continue operating, ensuring that user delivery fulfilment goals remain completed despite failures. For example, as a microservices-based system, even if the Order Processing System fails, the delivery tracking system will continue functioning, allowing for a seamless fulfilment experience between businesses and their customers.

This architecture enhances maintainability and scalability, as each component operates independently. Important components like Order Processing System and Real-Time Delivery Tracking System, can be scaled efficiently when utilization requirement is high. Therefore, the independence of each component allows the component to be replicated, ensuring that services can be allocated to specific or smaller user groups. This approach optimizes performance and minimizes points of failure, enhancing overall system reliability.

During the progression of the assignment, the observer pattern was clearly utilized in most of the systems created. From my viewpoint, this pattern is important to reduce server load by eliminating the need for client-side systems to repeatedly poll to the server or components for update. which repeatedly requesting server will severely impact the system scalability, performance and cause traffic congestions. For instance, during peak hours, a high volume of users will be actively using the system, where key components like restaurant and menu listing system, and real-time delivery tracking system will operate at maximum resource utilization. Therefore, Observer pattern played a role to ensure the central servers with the important service will push the relevant update to client system whenever the central server information is modified, eliminating unnecessary overhead of client requesting for update in the architectural solutions, directly reducing the server traffic and improve system efficiency.

Adapting pattern-based techniques and design pattern is crucial when designing a large-scale architectural solution. These approaches ensure that the system is well-structured, and that each component interacts seamlessly with others. Therefore, these pattern makes the system becomes more maintainable, scalable and adaptable to system usage and future modifications.